

값에는 정확한 posterior를, 선택에는 정확한 비용을: Binary Linear System을 위한 2단계 학습 branching

Exact Posteriors for Values, Exact Costs for Choices: Two-Stage Learned Branching for Binary Linear Systems

Gwang-Jong Ko*, Taesu Cheong †, In-Chan Choi † · School of Industrial and Management Engineering, Korea University · 2026-10-01 · 한국어판

초록

Binary linear system(BLS)은 0/1 행렬 A 에 대해 $Ax=b$ 를 만족하는 $x \in \{0,1\}^n$ 를 찾는 문제다. 이 문제에는 목적함수가 없고, 실제로 나타나는 instance에서는 linear relaxation이 주는 정보가 적으며, unit propagation은 탐색 tree의 root 근처에서 작동하지 않으므로 complete search의 성패는 branching 선택이 결정한다. 본 논문은 BLS의 depth-first search를 위한 branching heuristic을 두 개의 정확한 신호로 두 단계에 걸쳐 학습하는 방법을 제시한다. **Stage 1**은 어느 값을 먼저 시도할지 배운다: 모든 해를 열거할 수 있는 작은 instance에서는 conditional marginal $\Pr[x_j=1 \mid A, b, \text{partial assignment}]$ 가 정확하고, BLS를 partial assignment로 conditioning하는 것은 축소하는 것과 같으므로 축소 instance로 학습한 graph network 하나가 tree의 모든 node에서 예측한다. **Stage 2**는 어느 변수로 분기할지 배운다: 결정의 비용, 즉 그 subtree가 전개하는 node 수는 탐색이 정확히 산출하며, Stage 1 규칙에서 출발해 marginal network를 동결한 채 그 비용으로 actor-critic fine-tuning하면 branching 순서만 바뀐다. 배포 solver는 lattice reduction을 먼저 돌리고 남은 것에 guided search를 적용한다. wall-clock 예산을 맞추고 3 seed로 반복한 21×60 instance 100개에서 Stage 1은 LP 기반 branching보다 node를 5.5배 적게 쓰고 2.0~2.3배 빠르며, Stage 2는 추가로 node를 2.3~2.4배, 시간을 2.0~2.4배 줄이고($p < 0.001$) 학습 seed 간에 재현되며 본 적 없는 크기로 전이된다: 24×70 에서 fine-tuning된 policy는 30/30을 Stage 1보다 2.3~3.3배 빠르게 풀고 28×80 에서는 예산 내 해결율을 두 배로 올린다. ablation은 두 신호가 서로 대체 불가능함을 보인다: marginal network의 값 정확도를 마진이 사는 곳에서 올리면 tree가 커지고, 비용으로 조정된 policy는 network가 덜 확신하는 변수로 분기하면서 tree를 줄인다.

키워드: Binary linear systems · Learning to branch · Exact conditional marginals · Reinforcement learning · Tree search · Graph neural networks

1. 서론

Binary linear system(BLS)은 feasibility 문제

$$\text{find } x \in \{0,1\}^n \text{ such that } Ax=b, A \in \{0,1\}^{m \times n}, b \in \mathbb{Z}_{\geq 0}^m \quad (1)$$

이며 해집합을 S 로 쓴다. 레이저 기반 비파괴검사에서 이진 점유 영상이 정수 투영 집합을 정확히 재현해야 하는 복원 과제의 일반형이다: 스캔된 물체가 실재하므로 해는 존재하고 유일하지 않은 경우가 흔하며, 복원은 생산 라인의 속도를 따라가야 한다. 중요한 instance는 밀집하고 우변이 각 행 범위의 한가운데 근처에 있으며, 크기에 비해 불균형하게 어렵다.

이런 instance에서는 세 가지 성질이 표준 도구를 무력하게 만든다. 목적함수가 없으므로 branch-and-bound의 bound 기반 가지치기가 기댈 것이 없다. linear relaxation은 항상 feasible하지만 정보가 적다: relaxation polytope가 넓고 그 vertex가 $\{0,1\}^n$ 에서 멀어, vertex를 반올림해도 root에서 변수 값을 우연보다 조금 나은 정도로만 맞힌다. constraint solver의 주력인 unit propagation은 행이 포화될 때만 발동하는데 root에서는 결코, 그 근처에서도 거의 일어나지 않는다. 이 계열의 고전적 처방인 lattice basis reduction은 작은 instance를 바로 풀지만 크기가 커지면 작동을 멈추고, clause learning을 갖춘 complete solver는 빠르지만 branching 결정 자체를 싸게 맞히게 해주지는 않는다. complete search에 남은 것은 branching 결정의 품질이고, 그것이 본 논문의 대상이다.

branching 결정은 두 부분으로 이루어지며, 우리는 각 부분을 서로 다른 정확한 신호로 학습하는 것이 최선임을 보인다. 변수가 어느 값을 먼저 가져야 하는가는 해가 어디에 있는가의 질문이며, 열거 가능한 instance에서는 지금까지의 배정이 주어졌을 때 x_j 의 conditional marginal이 정확히 답한다. 어느 변수로 분기할 것인가는 그 뒤에 따르는 비용의 질문이며, subtree가 전개하는 node 수가 정확히 답하고 탐색이 backtrack하면서 그 수를 산출한다. 우리 방법의

Stage 1은 bipartite graph network를 첫째 신호로 학습한다. reduction 항등식(명제 1)이 같은 network를 tree의 모든 node에서 유효하게 만들고, marginal은 node의 변수를 "틀린 값이 subtree를 비우는 변수"(FORCED)와 "어느 값이든 해에 도달하는 변수"(OPEN)로 분할한다; 전자만이 backtrack 비용을 내고 그 비중은 depth에 따라 가파르게 커지므로 network는 tree 안에서 학습한다. Stage 2는 Stage 1 network를 동결한 채 둘째 신호로 branching 순서를 fine-tuning하여, 개선되는 것이 있다면 순서에만 귀속되게 한다. 배포 solver(그림 1)는 고전적 lattice 단계를 먼저 돌리고 남은 것에 guided search를 적용한다; network는 안내만 하므로 탐색은 완전하게 유지되고 잘못된 예측의 대가는 backtracking뿐이다.

기여. (1) complete search 안에서 정확한 conditional marginal로 학습한 BLS branching heuristic. target을 모든 node로 옮기는 reduction 항등식, FORCED/OPEN 분할, 그리고 in-tree 지도가 학습된 guidance를 relaxation rounding보다 낫게 만드는 요인이라는 발견(root 상태로 학습한 network는 더 못하다)은 새롭다. (2) posterior에서 유도된 규칙에서 출발해 순서만 학습하는 비용 기반 fine-tuning. 출발점과 동결된 marginal이 이득을 귀속시키고, 학습한 것보다 큰 instance로 전이하게 하는 요인이다. (3) 두 신호가 다른 것을 잡는다는 증거: 값 신호를 마진이 사는 곳에서 버리면 tree가 커지고, 비용으로 조정된 policy는 값 모델이 덜 확신하는 변수를 선호하면서 tree를 줄인다. (4) 통제된 평가: 크기 간 solution multiplicity 일치, wall-clock 예산 일치, 모든 branching 규칙에 동일한 탐색 루프, 평가 seed 3개, 학습 seed 2개, 100개 test set, CP-SAT·SCIP 비교, lattice reduction을 앞세운 배포 cascade.

2절은 관련 연구를 검토하고 이 문제에 남겨진 것을 정리하며, 3절은 문제와 reduction 항등식을, 4절은 방법을, 5절은 네 가지 질문으로 구성된 실험을 서술하고, 6절은 한계와 향후 연구로 맺는다.

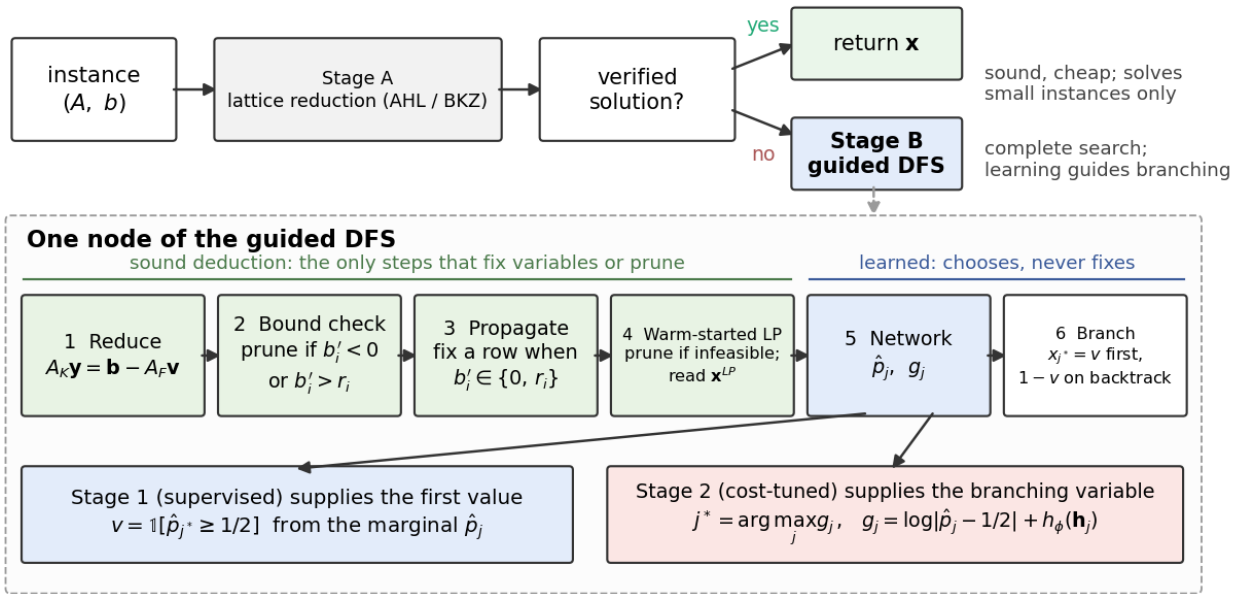


그림 1. 배포 solver. Stage A(lattice reduction)는 해를 찾으면 검증된 해를 반환하고, Stage B(guided depth-first search)가 나머지를 처리한다. node 안에서 1~4단계는 건전하며 변수를 확정하거나 가지를 치는 유일한 단계다; 5단계의 network는 선택만 하며, 값은 Stage 1에서, 변수는 Stage 2에서 온다.

2. 선행연구

해집합의 marginal을 branching 신호로. 결정에 동의하는 해의 비율로 분기하는 것은 학습 이전부터 있었다: counting-based search(Pesant et al., 2012)는 개별 제약 안에서 계산한 solution density가 최대인 변수-값으로 분기하고, belief-propagation-guided decimation(Montanari et al., 2007)은 근사 marginal로 가장 편향된 변수를 고정한다. NSNet(Li and Si, 2022)은 SAT 공식의 만족 배정을 열거해 얻은 정확한 marginal로 graph network를 학습하고 추정치를 반올림해 local search의 초기 배정으로 쓴다; MIP-GNN(Khalil et al., 2022), Neural Diving(Nair et al., 2020), predict-and-search(Han et al., 2023)는 수집한 준최적 해로부터 변수별 bias를 예측해 변수 고정·warm start·탐색 제한에 쓴다. Stage 1은 NSNet의 target을 공유하되 쓰는 자리와 방식이 다르다:

marginal은 conditional이며 reduction 항등식으로 complete search의 모든 node에 옮겨지고, 아무것도 확정하지 않은 채 branching을 안내하며, FORCED/OPEN 분할과 in-tree 대 root 통제 비교는 그 연구에 대응물이 없다.

Learning to branch. Khalil et al.(2016)과 Gasse et al.(2019)은 strong branching을 모방하며, 후자는 우리도 쓰는 bipartite 변수-제약 graph network를 쓴다; 모방은 순위가 줄이려는 tree 크기가 아니라 expert의 순위를 target으로 삼는다. tree 크기를 직접 최적화하는 것은 강화학습 연구의 한 줄기다: FMSTS(Etheve et al., 2020)는 node의 subtree 크기를 관측 가능한 Q-value로 쓰고 depth-first search에서는 모든 subtree를 최소화하면 전체 tree가 최소화됨을 보이며 from scratch로 학습한다; tree MDP(Scavuzzo et al., 2022)는 subtree 단위 credit assignment를 형식화하고, retro branching(Parsonson et al., 2023)은 회고적으로 추출한 subtree 경로에서 학습하며, SORREL(Feng and Yang, 2025)은 시연으로 초기화한다. Neuro#(Vaezipoor et al., 2021)은 #SAT solver의 residual formula 위의 graph network를 evolution strategies로 학습해 더 큰 instance로 일반화한다. Stage 2는 FMSTS의 credit assignment를 단순 policy gradient(Williams, 1992)로 쓴다; 더한 것은 출발점과 범위, 즉 정확한 marginal에서 유도된 규칙을 동결하고 branching 순서만 학습하는 것이다. 이 목적의 고전적 진술이 fail-first 원칙(Haralick and Elliott, 1980)과 impact-based search(Refalo, 2004)다. constraint programming에서는 value ordering을 해로부터(Chu and Stuckey, 2015) 또는 deep Q-learning으로(Cappart et al., 2021) 학습했다; 우리는 값은 정확한 posterior에서 취하고 변수만 학습한다.

Market split, lattice 방법, complete solver. Cornuéjols and Dawande(1999)의 market split 문제는 (1)과 같은 시스템에서 branch-and-bound의 약점을 드러내는 벤치마크다: $n=10(m-1)$ 개의 이진 변수 위에 등식 행 m 개, 계수는 $\{0, \dots, 99\}$ 에서 균등하게 뽑고 각 우변은 행 합의 절반으로 두므로, relaxation polytope가 넓고 vertex가 정수성에서 멀며 bounding은 거의 가지치지 못한다; branch-and-cut은 $m \approx 7$ 에서 멈춘다. Aardal et al.(2000)은 이 instance를 lattice 위에 재정식화해 풀었다: Aardal-Hurkens-Lenstra embedding은 해를 lattice의 가장 짧은 vector 중 하나로 만들고, LLL/BKZ reduction(Lenstra et al., 1982; Schnorr and Euchner, 1994)이 그것을 찾으며, 복원한 vector를 검증하므로 방법은 건전하다. lattice enumeration은 이 벤치마크에서 여전히 최고 수준이다: Wassermann(2025)은 QOBLIB market-split instance를 CPU 하나로 $m=14$ 까지 푼다. 우리 instance는 market split을 어렵게 만드는 재료 — 밀집한 행과 범위 한가운데의 우변 — 를 공유하고, 0/1 계수·섞어진 해·최대 80개 변수의 탐색 영역이라는 점에서 다르다; 따라서 lattice reduction은 BLS solver의 자연스러운 첫 단계이고, 우리 기여는 그것이 작동을 멈추는 곳에서 시작한다. clause learning을 갖춘 constraint solver(Marques-Silva and Sakallah, 1999; Ohrimenko et al., 2009)는 내려가면서 정보를 제조하며 우리가 비교하는 wall-clock 기준이다. 직접 satisfiability 예측은 기록이 약하고(Selsam et al., 2019; Li et al., 2023) message-passing network는 어떤 feasible/infeasible 쌍을 원리적으로 구분할 수 없다(Chen et al., 2023). 우리 방법의 두 학습 단계는 모두 amortized inference(Amos, 2023; Gershman and Goodman, 2014)의 사례다: marginal network는 변수마다 relaxation 하나를, cost head는 완료된 탐색을 대체한다.

이 접근들이 BLS에 남겨 둔 것. 네 가지 공백이 방법을 결정한다. 첫째, marginal 기반 방법은 marginal을 root에서 또는 개별 제약 안에서 한 번 추정된 뒤 변수를 확정한다; 해가 많은 feasibility 문제에는 모든 node에서 partial assignment에 조건부인 marginal이 필요하고, 추정이 틀려도 탐색이 완전하게 유지되는 사용법이 필요하다. 둘째, mixed-integer programming용 학습 branching은 목적함수 없이는 존재하지 않는 strong branching을 모방하거나, well-posed한 결정별 credit과 무작위가 아닌 출발점을 요구하는 from-scratch 학습을 한다; BLS에는 결정별로 정확한 비용 신호와 이미 좋은 규칙인 초기화가 필요하다. 셋째, lattice reduction은 건전하고 싸지만 coverage가 크기에 따라 붕괴하므로 대체가 아니라 탐색과의 결합이 필요하다. 넷째, network에 feasibility 판정을 맡길 수 없으므로 판정을 요구해서는 안 된다. 아래의 방법은 정확한 conditional marginal을 값 신호로, 정확한 subtree node 수를 비용 신호로, marginal 규칙을 비용 조정 policy의 출발점으로, network가 안내만 하는 cascade를 취한다.

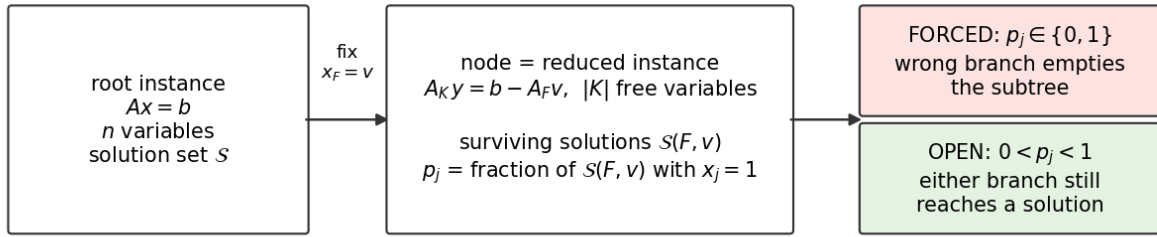
3. 문제 정의와 준비

3.1 Instance와 범위

우리는 (1)의 feasible instance를 다룬다: 모든 instance는 해를 심어 생성하므로 $S \neq \emptyset$ 이며 infeasibility 판별은 본 논문의 범위 밖이다. 구성은 2절의 market-split 방식에 0/1 계수를 적용한 것이다. 생성기는 밀집 무작위 A(각 원소가 확률 1/2로 1)를 뽑고 정확히 $n/2$ 개가 1인 무작위 x^* 를 심어 $b=Ax^*$ 로 두므로, 모든 우변은 행 범위의 한가운데, 즉 0/1 완성의 수가 가장 많고 propagation이 발동할 수 없는 곳에 놓인다. A마다 후보 planting 20개 중 relaxation polytope가 가장 넓은 것(무작위 선형 목적 4개가 돌려주는 vertex 간 평균 거리로 측정)을 남기므로 채택된 instance의 relaxation은 생성기가 만들 수 있는 만큼 무정보하다. 유일성은 강제하지 않고, $|S|=1$ 이면 모든 변수가 FORCED가 되어 과제가 vector 하나를 맞추는 것으로 퇴화하므로 크기마다 m 을 골라 solution multiplicity를 맞춘다(5.1절).

3.2 Conditioning은 reduction이다

Conditioning on a partial assignment is the same as reducing the instance (Proposition 1)



Only errors on FORCED variables cost a backtrack. Their share is 7.5% at the root and 88% at depth 8 (10×25 in-tree states), so the network is trained on reduced instances (in-tree states); labels are exact because S is enumerated.

그림 2. 명제 1과 그것이 유도하는 분할. $x_F=v$ 인 node는 더 작은 BLS $A_K y = b - A_F v$ 이고 그 conditional marginal은 그 instance의 통상적 marginal이다. 살아남은 모든 해가 일치하는 변수가 FORCED이며, 그 반대로 분기할 때만 backtrack 비용이 든다.

(1) 위의 depth-first search의 node는 고정 변수 집합 F 위의 partial assignment $x_F=v$ 이고 K 는 자유 변수다. $S(F,v)=\{x \in S: x_F=v\}$ 를 node와 일관된 해집합이라 하고

$$p_j = \Pr[x_j=1 \mid A, b, x_F=v] = |\{x \in S(F,v): x_j=1\}| / |S(F,v)|, j \in K \quad (2)$$

를 conditional marginal, 즉 planted-solution 생성기가 유도하는 S 위의 균등 prior 하의 posterior라 하자.

명제 1. A_F, A_K 를 F, K 로 인덱싱된 열 부분행렬이라 하면 $x \rightarrow x_K$ 는 $S(F,v)$ 에서 $\{y \in \{0,1\}^{|K|}: A_K y = b - A_F v\}$ 로의 전단사다. 따라서 원본 instance의 conditional marginal은 축소 instance의 통상적 marginal이다. 증명. $Ax = A_F x_F + A_K x_K$ 이고 $x_F=v$ 로 고정하면 $Ax=b \Leftrightarrow A_K x_K = b - A_F v$ 이며 x_F 가 결정되어 있으므로 전단사다. marginal 진술은 개수를 세면 따른다. $\square$

두 귀결이 있다(그림 2). 첫째, 하나의 architecture·라벨 생성기·학습 루프가 모든 node를 담당한다: depth- d 학습 상태는 $n-d$ 개 변수의 instance로 저장되고, 작은 instance로 학습한 network는 더 큰 instance의 탐색이 만들어내는 축소 instance에서 평가된다. 둘째, $p_j \in \{0,1\}$ 은 살아남은 모든 해가 x_j 에 일치한다는 뜻이고 반대로 분기하면 subtree가 빈다. 이런 변수를 FORCED, 나머지를 OPEN이라 부른다: OPEN 변수는 어느 branch로도 해에 도달하므로 첫 값이 틀려도 해를 포함한 subtree를 탐색하는 것 이상의 비용이 없다. FORCED 오류만이 backtrack 비용을 낳는다.

3.3 Node의 모습과 정확도 상한

작은 instance에서는 $S(F,v)$ 를 열거할 수 있으므로 해의 어떤 predictor도 넘을 수 없는 Bayes 최적 변수별 정확도를 계산할 수 있다:

$$\text{ceil}(F,v) = (1/|K|) \sum_{j \in K} \max(p_j, 1-p_j) \quad (3)$$

이는 살아남은 해의 다수 값으로 답할 때 달성된다. 그림 8a는 10×25 instance에서 node가 depth에 따라 어떻게 변하는지 보여준다: 살아남은 해집합은 root의 중앙값 21에서 depth 7이면 하나로 줄고, 자유 변수 중 FORCED 비중은

7.5%에서 depth 5의 61%, depth 8의 88%, depth 12의 98%로 오른다. root에서는 거의 어떤 결정도 비용을 낼 수 없고 상한 (3)은 OPEN 변수에 지배된다; depth 5부터는 대부분의 결정이 비용을 낼 수 있다. 이것이 branching heuristic이 잘해야 하는 영역이며, 아래에서 예측을 학습·평가하는 곳이다.

4. 제안 방법

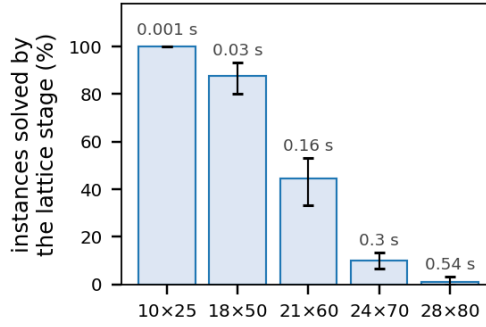
4.1 개요

solver는 두 단계다(그림 1). Stage A는 Aardal–Hurkens–Lenstra embedding 위의 lattice reduction으로, 성공하면 검증된 해를 돌려주는 건전한 고전 방법이다. Stage B는 학습된 network가 branching 결정을 안내하는 depth-first search다; 학습된 것은 결코 변수를 확정하거나 가지를 치지 않으므로 탐색은 완전하게 유지된다. network(그림 3)는 두 단계로 학습한다(그림 4): Stage 1은 marginal head를 열거한 instance의 정확한 conditional marginal에 맞추고, Stage 2는 동결된 trunk 위에 policy head를 더해 모든 branching 결정이 치르는 정확한 node 수로 fine-tuning한다.

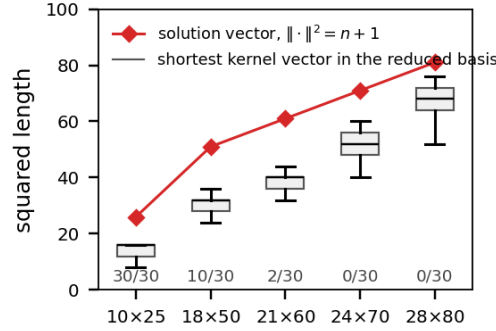
4.2 Lattice 단계

Aardal et al.(2000)의 embedding은 (1)을 Z^{n+1+m} 의 lattice로 옮긴다: 기저 행은 $j=1, \dots, n$ 에 대해 $(2e_j \mid 0 \mid N A_{:,j})$ 이고 마지막 행은 $(1 \mid 1 \mid Nb)$ 이며 N 은 큰 상수다. 해 x 에 대해 계수 $(x, -1)$ 의 정수 결합은 제약부가 $N(Ax-b)=0$, 표시부가 -1 , 변수부가 $\{\pm 1\}^n$ 인 vector, 즉 길이 제곱 $n+1$ 인 vector이고, 행을 어기는 vector는 N 의 배수를 지니므로 이것은 lattice에서 가장 짧은 축에 든다. 기저를 LLL로 축약한 뒤 block 12로 BKZ 축약하고(Lenstra et al., 1982; Schnorr and Euchner, 1994), 축약된 기저에서 그 형태의 행을 찾아 $x_j = (1 - w_j w_{n+1})/2$ 로 해독하고 $Ax=b$ 를 검증한 뒤 검증된 해만 반환한다; 무작위 열 순열을 최대 10개 시도한다. 이 단계는 instance당 $n=25$ 에서 0.001초, $n=60$ 에서 0.16초, $n=80$ 에서 0.54초가 든다.

coverage가 크기에 따라 떨어지는 이유는 축약된 기저에서 보인다(그림 5). 표시부 0, 제약부 0인 vector는 $Az=0$ 인 정수 kernel vector z 에 대응한다; 변수부가 $2z$ 이므로 0이 아닌 성분이 k 개인 kernel vector는 길이 제곱이 $4k$ 이고, kernel의 rank는 $n - \text{rank } A \approx 0.65n$ 이다. $n=25$ 에서 축약 기저의 가장 짧은 kernel vector는 길이 제곱 중앙값 16으로 해의 26보다 짧지만 모든 기저에 해 행이 들어 있다. $n=60$ 에서는 가장 짧은 kernel vector의 중앙값이 40으로 해의 61보다 짧고 기저 61행 중 39행이 kernel vector다; 해 행은 순열 하나로 30개 중 2개의 기저에서, 순열 10개로 33~53%의 instance에서 살아남는다. $n \geq 70$ 에서는 가장 짧은 kernel vector가 해보다 13~19만큼 짧고 해 행은 나타나지 않는다. reduction은 찾을 수 있는 가장 짧은 vector로 기저를 채우는데 $n \approx 60$ 부터 그것은 해가 아니라 kernel vector다. 이것이 우리가 중시하는 크기를 lattice가 아니라 학습된 탐색이 감당하는 이유다.



(a) coverage: 10 permutations, 3 seeds (range); time per instance above



(b) why it fails: kernel vectors become shorter than the solution (numbers at the bottom: bases with a solution row, one permutation)

그림 5. lattice 단계. (a) 열 순열 10개, seed 3개로 푸는 instance 비율(막대: 평균, whisker: 범위; 위의 숫자: instance당 시간 중앙값). (b) 축약 기저의 가장 짧은 kernel형 vector의 길이 제공(instance 30개, 순열 1개의 box)과 해 vector의 길이 제공 $n+1$; 아래의 숫자는 축약 기저에 해 행이 포함된 instance 수.

4.3 Guided depth-first search

lattice 단계가 닫지 못한 모든 instance는 알고리즘 1의 depth-first 절차로 처음부터 탐색한다. node는 partial assignment이며 명제 1에 따라 축소 instance $(A', b') = (A_K, b - A_F v)$ 로 다루고, r_i 는 A' 의 행 i 에 남은 자유 변수 수다.

알고리즘 1. guided depth-first search의 node 하나. 1~4단계는 건전하며 변수를 확정하거나 가지를 치는 유일한 단계다; 학습 컴포넌트는 5단계에만 들어간다.

단계	알고리즘 1: guided depth-first search의 node 하나
1	Reduce. (F, v) 를 pop; $A' = A_K$, $b' = b - A_F v$ 를 만든다. $r_i = 0$ 인 행은 버린다; 행이 남지 않으면 $Ax = b$ 를 검증하고 x 를 반환한다.
2	Bound check. 어떤 행에서 $b'_i < 0$ 또는 $b'_i > r_i$ 이면 node를 버린다.
3	Propagate. $b'_i = 0$ (그 행의 자유 변수를 0으로 고정) 또는 $b'_i = r_i$ (1로 고정)인 행이 있는 동안 고정을 적용한다; 모순이면 node를 버리고, 변수가 고정됐으면 축소 node를 push하고 돌아간다.
4	Relax. F 의 bound를 v 로 조인 단일 warm-start relaxation $\{Ax = b, 0 \leq x \leq 1\}$ 을 재최적화한다(이전 basis에서 dual simplex, 0.08 ms). infeasible이면 node를 버리고, 아니면 vertex x_K^{LP} 를 읽는다.
5	Predict. (A', b', x_K^{LP}) 의 bipartite graph 위에서 network를 한 번 forward하여 $j \in K$ 에 대한 p_j^* 와 g_j 를 얻는다(21x60 root에서 1.9 ms). $j^* = \arg \max_j g_j$, $v = 1[p_j^* \geq 1/2]$ 를 고른다.
6	Branch. 자식 $x_{j^*} = 1 - v$ 를 먼저, 자식 $x_{j^*} = v$ 를 나중에 push하여 v 를 먼저, backtrack 시 $1 - v$ 를 탐색한다.

세 가지 설계 선택이 이후에 중요하다. 0/1 행에서 propagation은 "이 행의 자유 변수 r_i 개 중 정확히 b'_i 개가 1"이라 읽히며 $b'_i \in \{0, r_i\}$ 일 때만 변수를 고정할 수 있으므로 우리 instance의 root 근처에서는 작동하지 않는다. relaxation은 node마다 다시 만들지 않는다: 원래 n 개 변수 위의 linear program 하나를 탐색 내내 유지하고 node를 bound 변경 집합으로 표현하므로, 자식이든 backtrack 후의 형제든 연속한 node는 bound 몇 개만 다르고 dual simplex는 부모 basis에서 재개한다; 이는 재구성보다 50배 싸며 Python 루프가 초당 약 700 node를 처리하게 하는 요인이다. 그 infeasibility 판정은 행들의 선형결합을 보므로 propagation보다 엄격히 강하고 network 앞의 마지막 건전 단계다. 그것이 돌려주는 vertex x^{LP} 가 network의 주 입력 특징이다.

4.4 Stage 1: 정확한 conditional marginal을 target으로

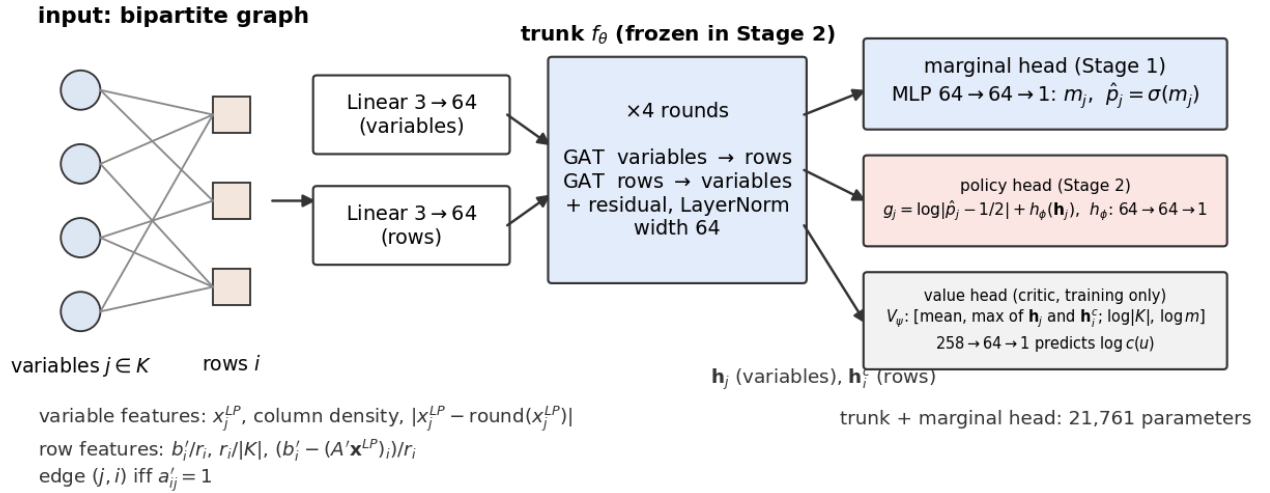


그림 3. network. 축소 instance가 변수당·행당 특징 3개를 지닌 bipartite graph로 들어가고, graph-attention 4 라운드의 trunk가 변수와 행마다 embedding을 만들며, head 셋이 그것을 읽는다. marginal head는 Stage 1에서, policy head는 동결된 trunk 위에서 Stage 2에서 학습하고, value head는 Stage 2의 critic으로만 쓰인다.

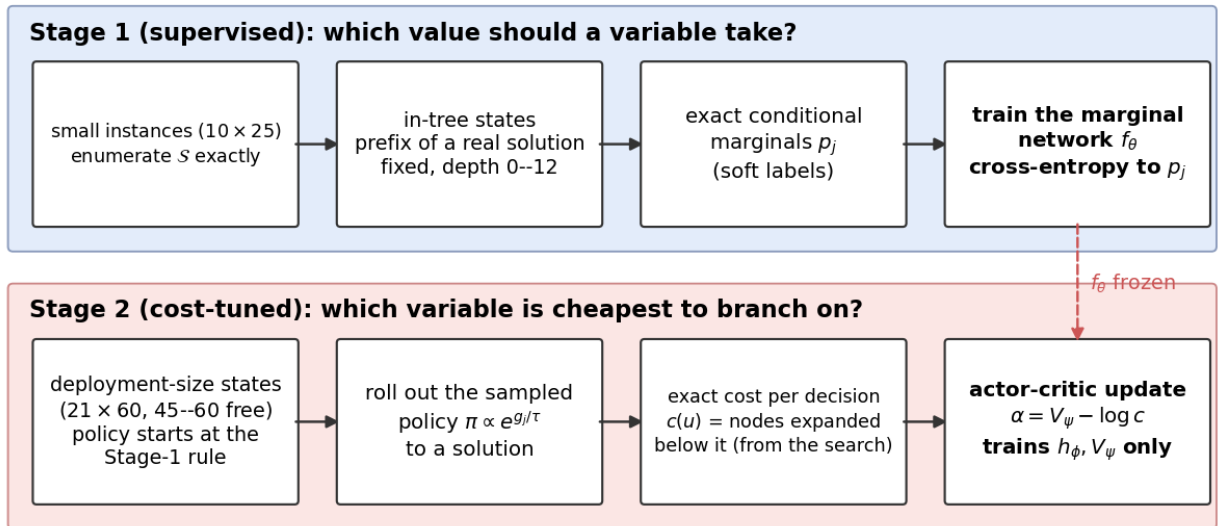


그림 4. 두 학습 단계. Stage 1은 열거 가능한 instance의 정확한 conditional marginal에서 어느 값을 배우고, Stage 2는 탐색이 산출하는 모든 결정의 정확한 node 수에서 어느 변수를 배운다. Stage 1 network는 동결된다.

Network. network f_θ (그림 3)는 graph attention(Velickovic et al., 2018)을 쓰는 bipartite 변수-제약 graph network로 Gasse et al.(2019)의 architecture 계열이다. 변수 node는 특징 3개(relaxation 값 x_j^{LP} , 열 밀도, fractionality $|x_j^{LP} - \text{round}(x_j^{LP})|$), 제약 node는 3개(우변 b'_i/r_i , 밀도 $r_i/|K|$, relaxation 잔차 $(b'_i - (A'x^{LP})_i)/r_i$)를 지니고, $a'_{ij}=1$ 인 곳에 간선이 있다. linear layer가 둘을 꼭 64로 올리고, attention 4 라운드가 변수 $\rightarrow$ 제약, 제약 $\rightarrow$ 변수 message를 번갈아 주고받으며(residual 연결, layer normalization), 2층 marginal head가 각 변수 embedding h_j 를 logit 하나로 사상한다: $p_j = \sigma(m_j)$; 파라미터는 총 21,761개. Stage 2에서 더하는 head 둘은 4.5절에 있다. architecture는 표준이고 기여는 target이다.

Target과 학습 상태. 열거 가능한 instance(10 $\times$ 25)에서 S 는 constraint solver의 all-solutions mode로 얻고 p_j 는 개수로 계산한다. 소진까지 완료된 열거만 인정한다; 해를 이미 모은 채 시간제한에 걸린 열거는 잘린 S 를 대입해 모든 라벨을 편향시킨다. 학습 상태는 도달 가능한 in-tree 상태: depth $d \in \{0, \dots, 12\}$ 를 균등하게 뽑고 LP 확신도 순서의 prefix를 무작위로 고른 실제 해의 값으로 고정된 뒤 축소 instance를 다시 열거해 자유 변수의 p_j 를 얻는다. network는

soft target에 대한 cross-entropy

$$L_1 = -(1/|K|) \sum_{j \in K} [p_j \log p_j^\wedge + (1-p_j) \log(1-p_j^\wedge)] \quad (4)$$

로 학습하며 이는 $p^\wedge = p$ 에서 정확히 최소화된다; p_j 자리에 심어진 해를 넣는 통상적 선택은 $|S| > 1$ 이면 잡음 섞인 대리 target이다.

Branching 규칙. node에서 network를 한 번 평가하고 Stage 1 규칙은 가장 확신하는 marginal의 다수 값으로 분기한다:

$$j^* = \operatorname{argmax}_{j \in K} |p_j^\wedge - 1/2|, \quad v = 1[p_{j^*}^\wedge \geq 1/2] \quad (5)$$

4.5 Stage 2: 정확한 subtree node 수를 목적함수로

규칙 (5)는 첫 자식이 해를 포함할 확률을 최대화한다. 첫 자식이 해를 포함하지 않을 때 탐색이 얼마를 지불하는지, 자식이 얼마나 작아지는지는 말하지 않는다; 둘 다 subtree의 성질이고 depth-first search는 그것을 정확히 산출한다. node u 에 대해 $c(u)$ 를 u 아래에서 해를 찾거나 subtree를 소진할 때까지 전개한 node 수라 하자; 탐색은 u 에서 backtrack해 나올 때 그것을 돌려준다. $c(u)$ 를 u 에서 내린 branching 결정의 비용으로 쓴다; $c(\text{root})$ 는 전체 탐색이 해에 도달하기까지 전개하는 node 수이며 실험 전체에서 보고하는 tree 크기 지표다. Stage 2는 $-\log c(u)$ 에 대해 branching 순서를 fine-tuning한다.

결정 과정. 상태는 propagation 후의 축소 instance, 행동은 자유 변수 j , 첫 값은 동결된 Stage 1 network의 $1[p_{j^*}^\wedge \geq 1/2]$ 로 유지한다. 각 결정이 자기 subtree를 가지므로 credit은 episode가 아니라 결정 단위로 배정된다 — sparse한 해결/미해결 보상에는 없는 성질이며 문제를 well-posed하게 만드는 성질이다(Etheve et al., 2020).

Policy와 value head. f_θ 의 동결된 trunk 위에 head 둘을 더한다(그림 3). policy logit은

$$g_j(s) = \log |p_j^\wedge - 1/2| + h_\phi(h_j) \quad (6)$$

이며 h_ϕ 는 마지막 층을 0으로 초기화한 2층 network라서 $\pi(j|s) \propto \exp(g_j/\tau)$ 는 규칙 (5)를 부드럽게 한 것에서 출발하고 그 argmax 는 정확히 그 규칙이다. value head $V_\psi(s)$ 는 mean·max pooling한 변수·제약 embedding과 $\log|K|$, $\log m$ 으로 $\log c$ 를 예측한다.

갱신. 매 epoch 자유 변수 45~60개인 학습 상태 400개를 뽑아 표집 policy($\tau=1$)를 해까지 전개하고 모든 결정을 정확한 비용과 함께 기록한다(epoch당 약 21,000 결정). epoch 안에서 표준화한 advantage $\alpha(u) = V_\psi(s_u) - \log c(u)$ 로

$$L_2 = \sum_u [-\alpha(u) \log \pi(j_u|s_u) - \lambda H(\pi(\cdot|s_u)) + (V_\psi(s_u) - \log c(u))^2], \quad \lambda=0.01 \quad (7)$$

이는 Monte-Carlo return을 쓰는 on-policy actor-critic 갱신이며(Williams, 1992) return이 정확하므로 bootstrapping이 필요 없다. ϕ , ψ 만 학습한다; θ 는 고정이므로 값을 고르고 FORCED/OPEN 분할을 정의하는 marginal은 변하지 않고 어떤 개선도 branching 순서만으로 귀속된다. critic은 규칙 (5) 하에서 기록한 결정으로 사전학습하며 그 held-out 순위 상관이 actor 갱신의 관문이다 — 비용 순위를 못 매기는 baseline은 advantage를 잡음으로 만든다. epoch는 학습 풀의 held-out 상태 아래 총 node 수로 고르며 평가 instance는 결코 쓰지 않는다. 배포 시 policy는 greedy다: $j^* = \operatorname{argmax}_j g_j(s)$.

4.6 Node당 비용

자유 변수 $|K|$ 개인 node에서 guided search는 warm-start relaxation 하나와 forward pass 하나를 지불하며, 21×60 에서 forward pass가 node 비용의 90% 이상이다. 같은 FORCED 변수를 탐지하는 건전 절차인 LP-probing(x_j 를 각 값으로 고정해 relaxation이 infeasible해지는 쪽을 가지치기)은 relaxation $|K|$ 개를 지불한다; 양쪽 모두 warm start로 측정된 비율은 $n=25$ 의 1.7배에서 $n=60$ 의 7.2배로 커진다. Stage 2의 head는 측정 가능한 비용을 더하지 않는다.

5. 실험

실험은 네 질문에 답한다: 학습된 branching이 같은 탐색 안에서 고전적 branching을 이기는가(5.2), 학습하지 않은 크기로 전이되는가(5.3), complete solver와의 거리는 얼마인가(5.4), 모든 구성요소가 필요한가(5.5).

5.1 설정

Instance. instance는 3.1절대로, 즉 0/1 계수와 심어진 해를 가진 market-split 구성으로 생성한다. 크기는 10×25 , 18×50 , 21×60 , 24×70 , 28×80 이며, 열거로 검증 가능한 곳에서는 해의 수 중앙값이 비슷하도록($|S|$: $n=25, 50, 60$ 에서 23, 19, 10; 60에서 27/30 검증), 열거가 완료되지 않는 $n \geq 70$ 에서는 제약 밀도 $m/n \approx 0.35$ 로 m 을 고른다. $n=100$ 에서는 CP-SAT가 20초 안에 해를 찾지 못하고 모든 전략이 실패하므로 $n=80$ 이 마지막 크기다. 평가는 크기당 30개와 21×60 100개 세트(생성기 seed 700000~700099, 30개 포함)를 쓴다; Stage 1은 별도의 10×25 instance 1,760개의 in-tree 상태 10,000개로, Stage 2는 별도의 21×60 풀 500개(seed 800000+, 400/100 분할)로 학습하며 그 held-out 100개가 epoch 선택 상태를 공급한다. 어떤 학습 풀과 어떤 평가셋의 겹침도 (A,b) 해시 기준 0이다.

학습. Stage 1: Adam, 학습률 10^{-3} , 20 epoch, batch 1, gradient clipping 1.0; CPU 1코어 20분; hyperparameter는 튜닝하지 않았고 validation split이 없다. Stage 2: 기록된 결정 37,275개(held-out 9,375개)로 critic 사전학습 후 갱신 (7)의 6 epoch; 학습 seed 2개. 21×60 에서 fine-tuning한 policy를 Stage 1+2 (60), 같은 프로토콜로 18×50 에서 fine-tuning한 둘째 policy를 Stage 1+2 (50)이라 하며 후자는 상향 전이를 검사한다.

표 1. branching arm. 모두 같은 depth-first 루프(알고리즘 1: node당 warm start relaxation 하나, 같은 propagation, 같은 예산)를 돌고 5단계만 다르다; 괄호는 학습 컴포넌트의 학습 크기. LP rule은 solver가 아니라 network 자리에 relaxation vertex를 넣은 같은 루프다.

arm	분기 변수	첫 값
random	무작위 자유 변수	무작위
LP rule	가장 정수에 가까운 relaxation 값: $\arg\max_j x_j^{LP} - 1/2 $	$\text{round}(x_j^{LP})$
Stage 1 (25)	가장 확신하는 marginal: $\arg\max_j p_j^\wedge - 1/2 $	$1[p_j^\wedge \geq 1/2]$
Stage 1+2 (60 / 50)	비용으로 조정된 점수: $\arg\max_j g_j$	$1[p_j^\wedge \geq 1/2]$

Arm과 solver. 표 1이 branching arm을 정의한다. random arm은 instance가 어려운지 확인하고, LP rule은 고전적 most-integral 규칙이자 branching 품질의 기준이다. CP-SAT(OR-Tools)와 SCIP은 같은 instance에서 단일 스레드, 같은 예산으로 complete solver로 돌려 wall-clock 기준으로 삼는다.

프로토콜과 지표. instance마다 두 지표를 보고한다: 탐색이 해에 도달하기까지 전개하는 node 수(4.5절의 $c(\text{root})$)는 branching 품질을 재며 relaxation vertex가 주어지면 탐색이 결정적이라 seed 불변이고, wall-clock 시간은 node당 비용까지 반영한다. 예산은 instance당 wall-clock(다섯 크기에 대해 60, 300, 600, 1,200, 1,200초)이며 프로세스 종료로 강제한다; 20코어 머신에서 instance 6개를 동시에 돌리고 시간에 민감한 코드는 전부 단일 스레드다(GPU 미사용). instance별 속도는 양측 부호검정으로 비교한다. 21×60 의 모든 수치는 평가 seed 3개로 반복하고, branching 비교에서는 lattice 단계를 끄고 cascade(5.5.4절)에서는 켜다. 열거와 CP-SAT는 OR-Tools, lattice reduction은 fpylll, relaxation은 Gurobi의 dual simplex다.

5.2 고전적 branching 대비 학습된 branching

표 2. 같은 warm start 루프에서의 branching 전략, lattice 단계 off, 3 seed(seed별 중앙값의 평균±표준편차; node 수는 seed 불변). Stage 1+2는 21×60에서 fine-tuning한 policy로 학습 seed 둘을 보고한다.

크기 (예산)	전략	해결	node 중앙값	시간 중앙값
10×25 (60초)	LP rule	30/30	46	0.03초
	Stage 1	30/30	24	0.07초
18×50 (300초)	LP rule	30/30	4,224	2.43±0.33초
	Stage 1	30/30	1,085	1.82±0.37초
	Stage 1+2	30/30	424	0.52±0.01초
21×60, 100개 (600초)	LP rule	100/100	79,609	41.2±1.9초
	Stage 1	100/100	13,386	16.5±0.6초
	Stage 1+2 (seed 0 / 1)	100/100	5,495 / 4,940	7.6 / 6.8초

표 2와 그림 6b가 주 결과다. Stage 1은 모든 크기에서 LP rule보다 작은 tree를 만들고(node 2.0배, 3.9배, 5.5배) 그 비율은 크기와 함께 커진다. 작은 tree가 시간을 사는 것은 tree가 forward pass를 상쇄할 만큼 커진 뒤다: 10×25에서는 Stage 1이 느리고, 18×50부터 빠르며(1.3~1.4배, 30개에서는 유의하지 않음), 21×60 100개 세트에서는 100개 중 70~71개에서 2.0~2.3배 빠르다($p<0.001$). Stage 2는 node당 비용은 그대로인 채 node를 추가로 2.3~2.4배, 시간을 2.0~2.4배 줄이며(64~68/100, $p\leq 0.007$) 이는 두 학습 seed와 모든 평가 seed에서 성립해 LP rule 대비 종단 4.8~5.9배다; fine-tuning된 policy는 18×50으로도 전이된다(node 2.6배 적음, 1.8~2.2배 빠름, $p\leq 0.016$). random branching은 20×50에서 300초 내 30개 중 하나도, 5.5.4절의 21×60 잔여도 하나도 풀지 못한다: instance는 어렵고 모든 건전 arm이 전부 닫으므로 비교는 node와 시간에 관한 것이다.

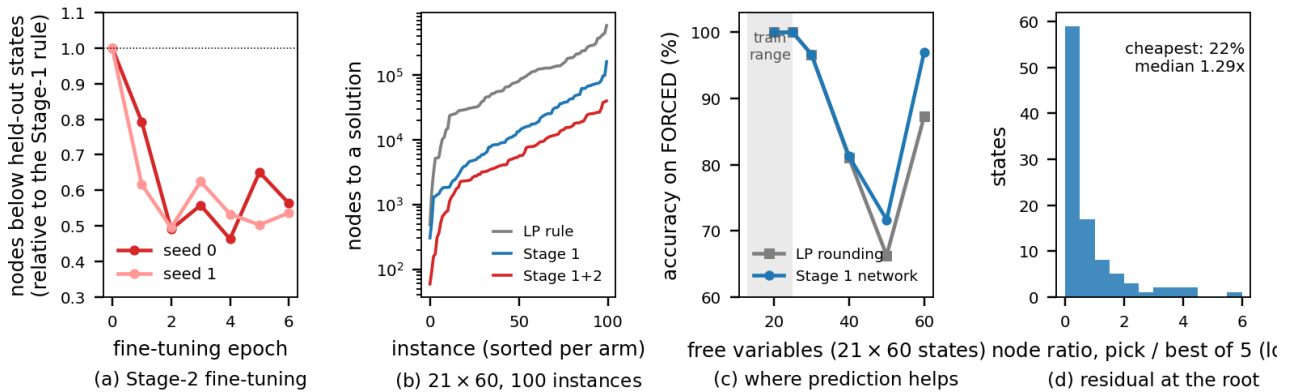


그림 6. (a) Stage 2 fine-tuning: held-out 상태 150개 아래에서 전개한 node 수(Stage 1 규칙 대비), 학습 seed 둘; 선택된 epoch는 4와 2. (b) 21×60 100개 세트에서 해까지의 node 수, arm별 정렬. (c) 21×60 in-tree 상태의 FORCED 변수 정확도(자유 변수 수별): Stage 1의 relaxation rounding 대비 마진은 학습 범위 밖에 있다. (d) root의 잔차: 확신도 규칙 선택 아래의 node 수를 후보 5개 중 가장 싼 것 대비로(held-out 상태 100개, 자유 변수 45~50).

5.3 크기 간 전이

표 3. 크기별 예산 내 해결 수, lattice off; 크기당 30개(21×60은 100개), seed 3개에서 다른 경우 범위, 28×80은 seed 1개. 괄호는 학습 크기.

크기 (예산)	LP rule	Stage 1 (25)	Stage 1+2 (60)	Stage 1+2 (50)
18×50 (300초)	30/30	30/30	30/30	30/30
21×60, 100개 (600초)	100/100	100/100	100/100	100/100
24×70 (1200초)	23~25/30	29/30	30/30	30/30
28×80 (1200초)	4/30	8/30	18/30	17/30

표 4. 크기별 해까지의 node 중앙값, lattice off(seed 불변; 표 3과 같은 instance, 28×80은 해결한 instance의 중앙값).

크기	LP rule	Stage 1 (25)	Stage 1+2 (60)	Stage 1+2 (50)
18×50	4,224	1,085	424	1,147
21×60, 100개	79,609	13,386	5,495	7,796
24×70	980,743	260,114	90,054	95,818
28×80	1,929,041	803,535	470,606	560,077

표 5. 같은 결과를 전이 방향으로 읽은 것: node 중앙값 비율, seed에 걸친 instance별 시간 비율 중앙값의 범위, 전이된 arm이 더 빠른 instance 수.

학습 → 테스트	대비	node	시간	우세
Stage 1: 25→50	LP rule	3.9배	1.3~1.4배	18/30
Stage 1: 25→60	LP rule	5.5배	2.0~2.3배	70~71/100
Stage 1: 25→70	LP rule	3.8배	0.7~1.1배	16~17/30
Stage 1+2: 60→50 (하향)	Stage 1	2.6배	1.8~2.2배	22~24/30
Stage 1+2: 50→60 (상향)	Stage 1	2.0배	1.7~1.9배	63~66/100
Stage 1+2: 60→70 (상향)	Stage 1	2.7배	2.3~3.3배	22~23/30
Stage 1+2: 50→70 (상향)	Stage 1	2.2배	2.1~2.7배	19~21/30
Stage 1+2: 50→50 (같은 크기)	Stage 1	0.9배	1.1~1.4배	16~18/30

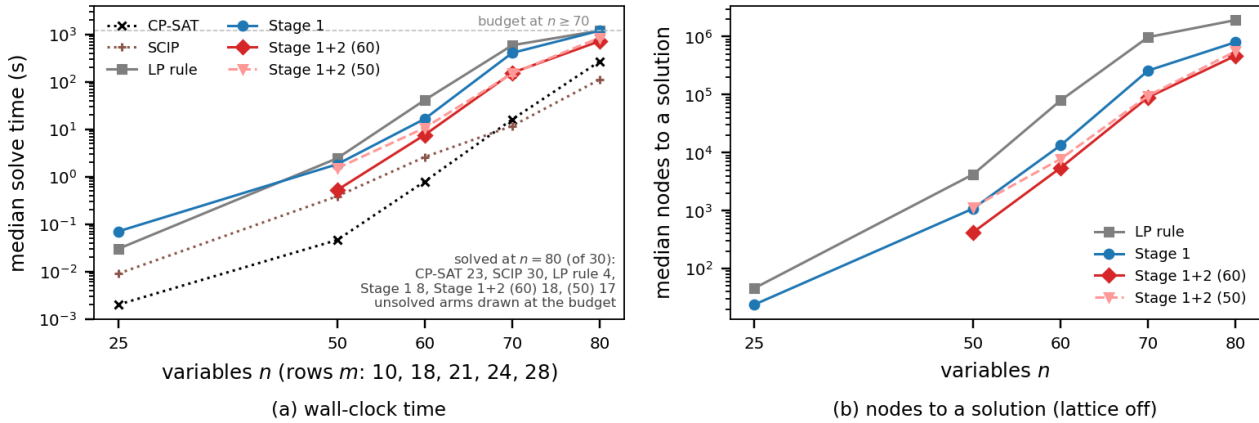


그림 7. 크기에 따른 스케일링. (a) branching arm과 complete solver의 wall-clock 시간 중앙값; $n \geq 70$ 에서 whisker는 seed 3개의 범위이고 $n=80$ 의 미해결 arm은 예산에 그리고 해결 수를 안에 표기했다. (b) 해까지의 node 중앙값, lattice off. 두 단계 모두 $n \leq 60$ 에서 학습했다.

두 단계 모두 양방향으로 전이된다(표 3~5, 그림 7). 10×25 에서만 학습한 Stage 1은 모든 크기에서 LP rule보다 node를 적게 쓰지만($n=50, 60, 70, 80$ 에서 3.9, 5.5, 3.8, 2.4배) tree가 커질수록 시간 이득은 줄어든다: 24×70 에서 29/30을 풀어 LP rule의 23~25/30보다 낮지만 둘 다 푼 instance에서는 더 빠르지 않다(0.7~1.1배) — 260,000-node tree는 260,000번의 forward pass를 지불하기 때문이다. Stage 2가 그 이득을 되살려 위로 옮긴다. 18×50 에서 fine-tuning한 policy는 21×60 에서 Stage 1보다 node가 2.0배 적고(100개 중 63~66개에서 빠름, $p < 0.001$); 두 fine-tuning policy 모두 24×70 에서 모든 seed에 30/30을 풀고 Stage 1보다 2.3~3.3배와 2.1~2.7배 빠르며 이는 학습 크기의 1.2~1.4배에서다; 28×80 에서는 1,200초 안에 30개 중 18개와 17개를 푸는데 Stage 1은 8개, LP rule은 4개다. 거기서 학습된 탐색의 창이 닫힌다: SCIP은 거기서 중앙 112초에 30/30을 푼다.

5.4 Complete solver와의 거리

표 6. 크기별 wall-clock 시간 중앙값, 단일 스레드, 동일 instance·예산; 전부 풀지 못한 arm의 해결 수는 표 3에 있다(CP-SAT는 28×80에서 23/30).

크기	CP-SAT	SCIP	LP rule	Stage 1	Stage 1+2 (60)	Stage 1+2 (50)
10×25	0.002초	0.009초	0.03초	0.07초	—	—
18×50	0.05초	0.39초	2.4초	1.8초	0.5초	1.5초
21×60, 100개	0.8초	2.6초	41초	16.5초	7.6초	10.7초
24×70	16초	12초	537~644초	358~457초	125~185초	135~165초
28×80	270초	112초	>1200초	>1200초	720초	852초

표 7. 21×60(30개)에서 CP-SAT와의 격차 분해. CP-SAT의 branch 카운터와 우리 node 카운터는 같은 단위가 아니므로(branch는 restart를 포함하고 clause learning이 건너뛴 것을 제외) 첫 행은 크기 차수의 비교일 뿐이다.

	CP-SAT	Stage 1+2
해까지의 branch / node 수 (중앙값)	18,827	4,040~8,873
초당 branch / node	19,782	≈700
시간 중앙값	1.16초	5.9~12.5초

표 6은 학습된 탐색을 같은 instance에서 complete solver 옆에 놓는다. CP-SAT는 18×50과 21×60에서 전체 방법보다 9~11배 빠르고, 24×70부터는 SCIP이 가장 빨라 10~15배 앞선다. 표 7이 격차를 분해한다: CP-SAT는 우리보다 작은 tree를 탐색하는 것이 아니라 node 하나를 약 28배 빨리 처리하며, clause learning과 restart를 갖춘 C++이고 우리 루프는 Python에서 돌며 node마다 90% 이상을 network의 forward pass에 쓴다. 부족분은 branching 품질이 아니라 node당 처리량이다; compiled inference 경로와 clause learning이 그것을 닫는지는 여기서 보이지 않았다.

5.5 Ablation

5.5.1 Stage 1 신호: in-tree 라벨, 그 다음 graph 구조

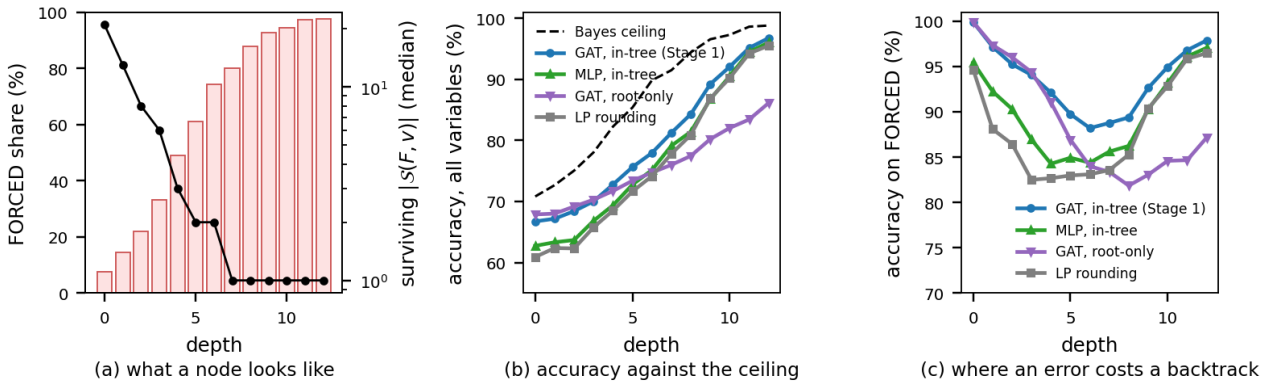


그림 8. 10×25 instance의 held-out in-tree 상태 2,640개에서 정확한 posterior 대비 예측, depth별. (a) 자유 변수 중 FORCED 비중과 살아남은 해 수의 중앙값. (b) 모든 자유 변수에 대한 변수별 정확도와 Bayes 상한 (3). (c) 오류가 backtrack 비용을 낳는 FORCED 변수에서의 정확도. 네 predictor는 라벨·손실·상태를 공유한다: Stage 1 network(in-tree로 학습한 GAT), root 상태만으로 학습한 같은 GAT, in-tree로 학습한 제약 집계 특징의 graph-free MLP, relaxation rounding.

표 8. 10×25 instance의 held-out in-tree 상태에서 오류가 backtrack 비용을 낳는 FORCED 변수의 정확도; 정답은 참조 해이고 상한은 정의상 100%다. 모든 자유 변수에 대한 정확도는 그림 8b에 있다.

predictor	전체 depth	depth 8
GAT, in-tree (Stage 1)	93.2%	89.4%
MLP, in-tree	89.5%	86.2%
GAT, root만 (통제)	88.0%	81.9%
LP rounding	87.9%	85.3%
FORCED 변수 비중	57.2%	87.9%

표 8과 그림 8이 Stage 1 신호를 작동하게 하는 요인을 분리한다. in-tree 라벨이 먼저다. 같은 architecture·라벨·optimizer·상태 수로 root 상태만 학습한 network는 root에서는 in-tree network보다 낮지만(모든 자유 변수에서 67.9% 대 66.7%) depth 4부터 못하고, 전체 depth에서는 모든 변수에 대해 relaxation rounding 아래이며(75.5% 대 76.5%) depth 8의 FORCED 변수에서는 3.4%p 아래다. root의 결정은 거의 비용을 낳지 않으므로 root 정확도는 branching heuristic에 잘못된 목적이다. graph 구조는 그 다음이다. 같은 변수별 특징에 인접 행의 mean·max 집계를 더한 graph-free MLP를 같은 in-tree 상태로 학습하면 FORCED 변수에서 89.5%로 GAT의 93.2%보다 낮고, 동일 2,000/800 분할에서 정확한 marginal까지의 L_1 거리는 0.225 대 0.151이다. 라벨 효과(FORCED 변수에서 5~7%p)가 구조 효과(3~4%p)보다 크고, 모든 depth에서 고전 규칙 위에 머물려면 둘 다 필요하다. 학습 크기 안에서는 학습 상태의 depth 범위가 중요하지 않다(depth 0~6, 6~12, 0~12가 79.3, 79.9, 79.9%).

배포 크기에서 마진이 어디 사는지는 21×60 in-tree 상태의 그림 6c에 있다: 학습 범위(자유 변수 13~25) 안에서는 network와 rounding이 구별되지 않는데, 제약 밀도 $m/|K| \geq 0.8$ 에서는 relaxation이 이미 모든 FORCED 변수에서 정수이기 때문이다; 마진 전부(+5%p)는 학습 범위 밖인 자유 변수 50~60개, relaxation이 가장 약한 곳에서 생긴다. 21×60 탐색 질의 80%가 거기에 있다.

5.5.2 두 신호는 서로 대체 불가능하다

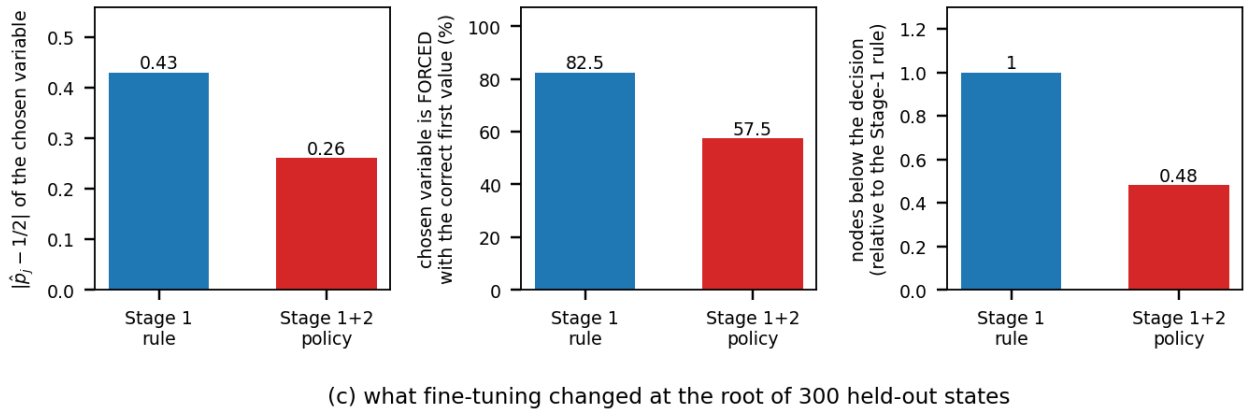
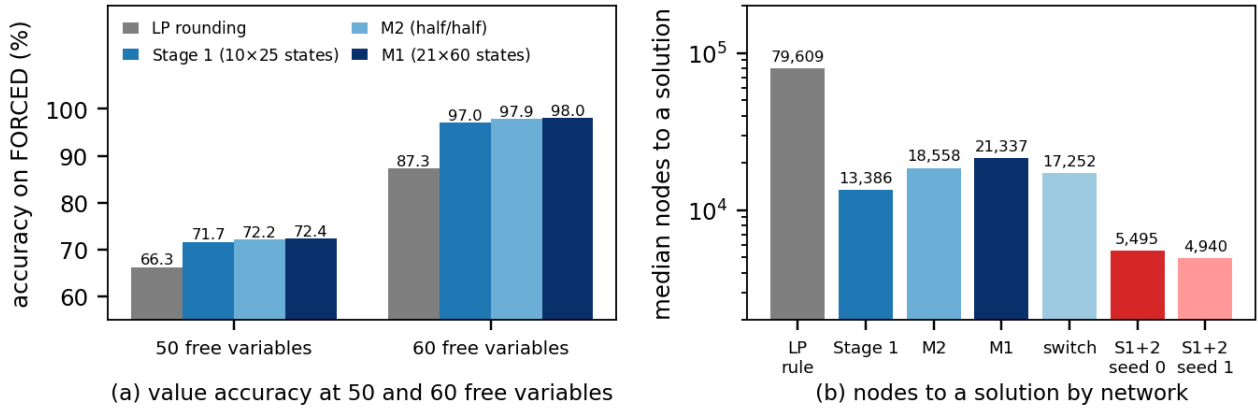


그림 9. 값 정확도와 tree 크기는 독립적으로 움직인다. (a) 자유 변수 50개와 60개인 21x60 in-tree 상태의 FORCED 변수 정확도: relaxation rounding, 10x25 상태로 학습한 Stage 1 network, 같은 architecture를 21x60 상태(M1)나 반반 혼합(M2)으로 학습한 것. (b) 100개 세트에서 해까지의 node 중앙값: 값 정확도가 높은 network가 node를 더 많이 전개하고 비용으로 조정된 policy는 덜 전개한다. (c) held-out 상태 300개의 root에서 fine-tuning된 policy는 덜 확신하는 변수, FORCED이면서 맞는 경우가 더 적은 변수를 고르고 그 결정 아래의 node 수는 절반이다.

값 신호를 버리면 tree가 커진다. 그림 6c는 명백한 처방을 시사한다: Stage 1을 마진이 사는 곳에서 학습하라. 21x60 학습 풀에서 자유 변수 30~60개인 in-tree 상태 2,691개를 만들어(정확한 marginal; 자유 변수 55개 이하인 상태는 전부 90초 내 열거) 같은 architecture를 그것만으로(M1), 10x25 상태와 반반 혼합으로(M2) 학습했다. 자유 변수 50개와 60개에서 FORCED 정확도는 약 1%p 오른다(71.7→72.4%, 97.0→98.0%; 그림 9a). tree는 커진다: M1은 100개 세트에서 node 중앙 21,337개를 전개해 Stage 1의 13,386보다 많고(더 적은 경우 38/100뿐, $p=0.021$), M2는 18,558, 자유 변수 50개 이상인 node를 M1로, 나머지를 Stage 1로 보내는 switch는 아무것도 회복하지 못한다(17,252; 그림 9b). 이유는 결정이 있는 곳이다: 결정의 2/3가 자유 변수 25~34개에서 일어나고, 그런 작고 촘촘한 상태에서 M1의 최고 확신 변수가 FORCED이면서 맞는 경우는 77.6%로 Stage 1의 94.4%보다 낮다(실제 마진과의 순위 상관 0.12 대 0.36). tree 상단에서 조금 낮고 내부에서 훨씬 못한 network는 탐색을 느리게 한다; 값 신호의 학습 분포는 지렛대가 아니다.

확신하는 선택은 드물게만 가장 싸다. 자유 변수 45~50개인 held-out 상태 100개에서 root를 가장 확신하는 변수 5개 각각으로 강제하고 아래는 규칙 (5)로 돌렸다(그림 6d). 규칙 자신의 선택이 5개 중 가장 싼 경우는 22%; 최선 후보 대비 node 수 비율 중앙값 1.29, 평균 2.97; 2배 이상 싼 후보가 있는 상태 24%; root 최선 선택만으로도 이 상태들의 총 node 55%가 제거된다. 잔차는 어느 변수인가에 관한 것이고 크다.

비용 신호는 확신에서 멀어진다. 동결 trunk 위의 critic은 log c와 held-out Spearman 상관 0.80에 이르고(상태 크기만 아는 baseline 0.70; 자유 변수 35~44 구간에서 0.68 대 0.38), trunk를 풀면 0.02가 더해질 뿐이라 쓰지 않는다. fine-tuning 6 epoch으로 held-out 상태 아래의 node 수는 Stage 1 규칙의 0.46(seed 0, epoch 4 선택)과 0.50(seed 1, epoch 2)이 되고 policy entropy는 1.7에서 0.1 아래로 떨어진다(그림 6a). held-out 상태 300개에서 fine-tuning된 policy는 95%에서 규칙 (5)와 다른 root 변수로 분기한다; 그 선택은 대개 15번째로 확신하는

변수이고($|p^{-1/2}|$ 0.26 대 0.43) FORCED이면서 첫 값이 맞을 확률도 낮다(57.5% 대 82.5%; 그림 9c). 값 기준으로는 fine-tuning된 선택이 모든 면에서 나쁘지만 tree는 절반 이하다. 이득은 root 결정만의 성질이 아니다: root만 fine-tuning 선택으로 강제하고 아래를 규칙 (5)로 돌리면 node 비율 중앙값 0.93(22/46에서 더 썸)이라 개선되는 것은 결정 수열이다. 두 학습 seed는 많은 node에서 다른 변수를 가르면서 비슷한 node 수에 도달하는데, 특정 해가 아니라 목적함수가 학습됐다면 기대되는 바다. branch가 맞을 확률과 틀렸을 때의 비용은 다른 목적함수이고 탐색이 최소화하는 것은 후자다.

5.5.3 Stage 2가 중요한 곳

표 9. 평가 크기별로 본 비용 조정 policy의 Stage 1 규칙 대비 이득: node 중앙값 비율과 seed에 걸친 instance별 시간 비율 중앙값의 범위.

조건	node	시간
18×50, 18×50에서 학습 (10^3 node의 tree)	0.9배	1.1~1.4배
18×50, 21×60에서 학습	2.6배	1.8~2.2배
21×60, 21×60에서 학습	2.3~2.4배	2.0~2.4배
24×70, 21×60에서 학습	2.7배	2.3~3.3배
28×80, 21×60에서 학습	해결 8/30 → 18/30	

fine-tuning된 policy는 tree가 작을 때 자기 학습 크기에서 이득이 없고(18×50: 1,147 대 1,085 node) 이득은 tree와 함께 커진다(표 9). 그림 6c와 일관되게, 배운 것은 크고 약하게 제약된 상태에서 중요하고 그런 상태는 큰 tree에만 있다. 18×50 아래에서는 Stage 1 규칙으로 충분하고, 21×60 위에서는 비용 신호가 학습된 탐색을 LP rule보다 앞서게 하는 유일한 요소다.

5.5.4 Lattice 단계와 잔여

표 10. 크기별 lattice 단계의 coverage: seed 3개(seed가 열 순열을 정함) 각각에서 푼 instance 수와 실패한 시도를 포함한 instance당 시간 중앙값.

크기	lattice 단계가 푼 수	instance당
10×25	30개 중 30, 30, 30	0.001초
18×50	30개 중 27, 28, 24	0.03초
21×60 (30)	30개 중 16, 14, 10	0.16초
21×60 (100)	100개 중 49, 38, 45	0.16초
24×70	30개 중 3, 4, 2	0.30초
28×80	30개 중 0, 1, 0	0.54초

표 11. lattice 단계가 남긴 21×60 잔여(seed 3개 합산 50개; 한 seed 안의 모든 arm은 동일한 잔여를 받는다): 600초 내 해결 수와 그 위의 시간 중앙값, seed 범위.

잔여에서의 arm	해결	시간 중앙값
random	0/50	>600초
LP rule	50/50	52.9~65.1초
Stage 1	50/50	14.2~20.2초
Stage 1+2	50/50	7.3~11.7초

표 10과 표 11은 학습된 탐색을 배포되는 자리에 놓는다. lattice 단계는 $n=25$ 에서 전부, $n=50$ 에서 대부분, $n=60$ 에서 $1/3 \sim 1/2$ 을 달고 그 위에서는 거의 달지 못한다; 실패한 시도의 비용은 instance당 최대 0.7초로 뒤따르는 탐색에 비해 무시할 만하다. 21×60 잔여는 진정으로 어렵다 — random branching은 하나도 달지 못하고 모든 건전 전략이 전부 달으며 차이는 시간이다: Stage 1은 잔여에서 LP rule보다 3.0배 빠르고($40/50$, $p=2 \times 10^{-5}$) Stage 2는 seed의 잔여에 따라 추가로 1.2~2.7배다. $n \geq 70$ 에서는 잔여가 전체 세트이고 표 3과 표 6이 적용된다: 24×70에서 fine-tuning된 policy는 30/30을 달는데 lattice는 2~4개, LP rule은 23~25개를 달는다.

6. 결론

branching 결정은 두 질문을 던지고, 본 논문은 각각의 정확한 답으로 학습한다. 정확한 conditional posterior — 해를 열거할 수 있는 곳에서 계산되고 reduction 항등식으로 모든 node에 옮겨지는 — 는 탐색에 어느 값을 시도할지 말해주고 어떤 예측이 비용을 낳는지 특정한다; 결정 아래의 정확한 node 수 — 탐색이 backtrack하며 산출하는 — 는 어느 변수로 분기할지 말해준다. lattice 단계 뒤에 배치되어 어려운 feasible binary linear system에서 첫째 신호는 21×60 instance 100개에서 LP 기반 branching을 2.0~2.3배 앞서고, 둘째는 첫째를 건드리지 않는 fine-tuning으로 2.0~2.4배를 더하며 seed 간에 재현되고 첫째 신호만으로는 고전 규칙을 더는 이기지 못하는 24×70 과 28×80 으로 상향 전이된다. fine-tuning된 policy가 posterior가 덜 확신하는 변수를 선호한다는 것과, posterior를 그 마진이 사는 곳에서 버리면 tree가 커진다는 것은 같은 사실을 두 번 본 것이다: 맞을 확률과 틀렸을 때의 비용은 다른 목적함수이고 탐색이 최소화하는 것은 후자다.

두 한계가 결과를 제한한다. 정확한 지도가 Stage 1을 제한한다: marginal은 S의 열거를 요구하고 이는 $n \approx 60$ 을 넘으면 완료되지 않으므로 Stage 1은 큰 크기로 전이되지만 거기서 학습할 수 없고, 학습된 탐색의 유용한 창은 $n \approx 80$ 에서 닫힌다. 그리고 시스템은 wall-clock 시간에서 complete solver와 경쟁하지 못한다: CP-SAT는 $n \leq 60$ 에서 9~11배, SCIP은 $n \geq 70$ 에서 10~15배 빠르며, 이는 tree 크기가 아니라 forward pass가 지배하는 Python 루프의 node당 처리량의 부족분이다.

둘 다 다음 단계를 가리킨다. Stage 2의 비용 신호는 열거를 요구하지 않으므로 열거가 실패하는 크기에서도 branching 순서를 fine-tuning할 수 있고, 결정 아래의 node 수는 subtree가 해를 포함하든 아니든 정의되므로 본 논문이 제외한 infeasible instance에서도 가능하다. 처리량 격차는 compiled inference 경로와, complete solver는 쓰지만 우리 탐색에는 없는 clause learning·restart를 요구한다. ablation 둘이 남아 있다 — 비용 조정 변수 선택과 LP rounding 값의 결합, 그리고 propagation이나 relaxation 단계를 뺀 탐색 루프 — 그리고 어느 신호든 다른 constraint 계열로 전이되는지는 미검증이다.

참고문헌

- Aardal, K., Hurkens, C. A. J., and Lenstra, A. K. (2000). Solving a system of linear Diophantine equations with lower and upper bounds on the variables. *Mathematics of Operations Research*, 25(3):427–442.
- Aardal, K., Bixby, R. E., Hurkens, C. A. J., Lenstra, A. K., and Smeltink, J. W. (2000). Market split and basis reduction: Towards a solution of the Cornuéjols–Dawande instances. *INFORMS Journal on Computing*, 12(3):192–202.
- Amos, B. (2023). Tutorial on amortized optimization. *Foundations and Trends in Machine Learning*, 16(5):592–732.
- Cappart, Q., Moisan, T., Rousseau, L.-M., Prémont-Schwarz, I., and Cire, A. A. (2021). Combining reinforcement learning and constraint programming for combinatorial optimization. In *AAAI Conference on Artificial Intelligence*.
- Chen, Z., Liu, J., Wang, X., Lu, J., and Yin, W. (2023). On representing mixed-integer linear programs by graph neural networks. In *International Conference on Learning Representations*.
- Chu, G. and Stuckey, P. J. (2015). Learning value heuristics for constraint programming. In *CPAIOR*.
- Cornuéjols, G. and Dawande, M. (1999). A class of hard small 0-1 programs. In *Integer Programming and Combinatorial Optimization*, pages 284–293.
- Etheve, M., Alès, Z., Bissuel, C., Juan, O., and Kedad-Sidhoum, S. (2020). Reinforcement learning for variable selection in a branch and bound algorithm. In *CPAIOR*, pages 176–185.
- Feng, S. and Yang, Y. (2025). SORREL: Suboptimal-demonstration-guided reinforcement learning for learning to branch. In *AAAI Conference on Artificial Intelligence*.
- Gasse, M., Chételat, D., Ferroni, N., Charlin, L., and Lodi, A. (2019). Exact combinatorial optimization with graph convolutional neural networks. In *Advances in Neural Information Processing Systems*, volume 32.
- Gershman, S. J. and Goodman, N. D. (2014). Amortized inference in probabilistic reasoning. In *Proceedings of the Annual Meeting of the Cognitive Science Society*, volume 36.
- Han, Q., Yang, L., Chen, Q., Zhou, X., Zhang, D., Wang, A., Sun, R., and Luo, X. (2023). A GNN-guided predict-and-search framework for mixed-integer linear programming. In *International Conference on Learning Representations*.

- Haralick, R. M. and Elliott, G. L. (1980). Increasing tree search efficiency for constraint satisfaction problems. *Artificial Intelligence*, 14(3):263–313.
- Khalil, E. B., Le Bodic, P., Song, L., Nemhauser, G., and Dilkina, B. (2016). Learning to branch in mixed integer programming. In *AAAI Conference on Artificial Intelligence*.
- Khalil, E. B., Morris, C., and Lodi, A. (2022). MIP-GNN: A data-driven framework for guiding combinatorial solvers. In *AAAI Conference on Artificial Intelligence*.
- Lenstra, A. K., Lenstra, H. W., and Lovász, L. (1982). Factoring polynomials with rational coefficients. *Mathematische Annalen*, 261(4):515–534.
- Li, Z. and Si, X. (2022). NSNet: A general neural probabilistic framework for satisfiability problems. In *Advances in Neural Information Processing Systems*, volume 35.
- Li, Z., Guo, J., and Si, X. (2023). G4SATBench: Benchmarking and advancing SAT solving with graph neural networks. *Transactions on Machine Learning Research*.
- Marques-Silva, J. P. and Sakallah, K. A. (1999). GRASP: A search algorithm for propositional satisfiability. *IEEE Transactions on Computers*, 48(5):506–521.
- Montanari, A., Ricci-Tersenghi, F., and Semerjian, G. (2007). Solving constraint satisfaction problems through belief propagation-guided decimation. In *Allerton Conference on Communication, Control, and Computing*.
- Nair, V., Bartunov, S., Gimeno, F., et al. (2020). Solving mixed integer programs using neural networks. *arXiv:2012.13349*.
- Ohrimenko, O., Stuckey, P. J., and Codish, M. (2009). Propagation via lazy clause generation. *Constraints*, 14(3):357–391.
- Parsonson, C. W. F., Laterre, A., and Barrett, T. D. (2023). Reinforcement learning for branch-and-bound optimisation using retrospective trajectories. In *AAAI Conference on Artificial Intelligence*.
- Pesant, G., Quimper, C.-G., and Zanarini, A. (2012). Counting-based search: Branching heuristics for constraint satisfaction problems. *Journal of Artificial Intelligence Research*, 43:173–210.
- Refalo, P. (2004). Impact-based search strategies for constraint programming. In *Principles and Practice of Constraint Programming (CP)*, pages 557–571.
- Scavuzzo, L., Chen, F. Y., Chételat, D., Gasse, M., Lodi, A., Yorke-Smith, N., and Aardal, K. (2022). Learning to branch with tree MDPs. In *Advances in Neural Information Processing Systems*, volume 35.
- Schnorr, C. P. and Euchner, M. (1994). Lattice basis reduction: Improved practical algorithms and solving subset sum problems. *Mathematical Programming*, 66:181–199.
- Selsam, D., Lamm, M., Bünz, B., Liang, P., de Moura, L., and Dill, D. L. (2019). Learning a SAT solver from single-bit supervision. In *International Conference on Learning Representations*.
- Vaezipoor, P., Lederman, G., Wu, Y., Maddison, C., Grosse, R. B., Seshia, S. A., and Bacchus, F. (2021). Learning branching heuristics for propositional model counting. In *AAAI Conference on Artificial Intelligence*.
- Velickovic, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., and Bengio, Y. (2018). Graph attention networks. In *International Conference on Learning Representations*.
- Wassermann, A. (2025). Solving the market split problem with lattice enumeration. *arXiv:2508.08702*; *Mathematical Programming Computation*, to appear.
- Williams, R. J. (1992). Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3–4):229–256.